

Notas Práticas e Exercícios 5

Disciplina: Programação Orientada a Objetos

{*Anotações para uso pessoal}

1 Exercício Exceções

Implementação do exercício descrito nos slides da aula de exceções.

```
1 public class OperacaoInvalidaException extends Exception {
2     public OperacaoInvalidaException(String message) {
3         super(message);
4     }
5 }
6
7 public class Calculator {
8     public double calcular(double a, double b, char operacao) throws
9         ArithmeticException, OperacaoInvalidaException {
10         switch (operacao) {
11             case '+':
12                 return a + b;
13             case '-':
14                 return a - b;
15             case '*':
16                 return a * b;
17             case '/':
18                 if (b == 0) {
19                     throw new ArithmeticException("Divisão por zero.");
20                 }
21                 return a / b;
22             default:
23                 throw new OperacaoInvalidaException("Operação inválida: " +
24                     operacao);
25         }
26     }
27 }
28
29 public class ExemploCalculadora {
30     public static void main(String[] args) {
31         double a = 10;
32         double b = 5;
33         char operacao = '#'; // Operação inválida
34
35         Calculator calculator = new Calculator();
```

```

34
35     try {
36         double resultado = calculator.calcular(a, b, operacao);
37         System.out.println("Resultado: " + resultado);
38     } catch (OperacaoInvalidaException e) {
39         System.out.println("Erro: " + e.getMessage());
40     } catch (ArithmeticException e) {
41         System.out.println("Erro: " + e.getMessage());
42     }
43 }
44 }

```

2 Exercícios Práticos

1. Exercício: Interface, Classe Abstrata e Sistema de Reservas de Passagens Aéreas

Crie uma interface chamada `Passagem` com os seguintes métodos:

- `String getCodigo();`
- `double getPreco();`
- `LocalDateTime getDataHora();`
- `String getOrigem();`
- `String getDestino();`

Crie uma classe abstrata chamada `PassagemAbstrata` que implemente a interface `Passagem` e adicione os seguintes atributos e métodos:

- Atributo `String codigo;`
- Atributo `double preco;`
- Atributo `LocalDateTime dataHora;`
- Atributo `String origem;`
- Atributo `String destino;`
- Método abstrato `double calcularPrecoFinal();`

Implemente duas classes que herdem de `PassagemAbstrata`: `PassagemEconomica` e `PassagemExecutiva`. Ambas as classes devem implementar o método `calcularPrecoFinal()` de acordo com as regras abaixo:

- `PassagemEconomica`: O preço final deve ser igual ao preço base.
- `PassagemExecutiva`: O preço final deve ser o preço base multiplicado por 1.5.

Crie uma classe chamada `SistemaReservas` com os seguintes atributos e métodos:

- Atributo `List<Passagem> passagens;`
- Método `void adicionarPassagem(Passagem passagem);`
- Método `void removerPassagem(String codigo);`
- Método `void listarPassagens();`
- Método `Passagem buscarPassagem(String codigo);`

- Método `double calcularPrecoFinal(String codigo);`

Os métodos devem interagir com a lista de passagens, permitindo adicionar, remover, listar, buscar e calcular o preço final das passagens de acordo com suas regras.

2. Exercício: Sistema de Cadastro de Alunos com Exceções

Crie uma classe chamada `Aluno` com os seguintes atributos e métodos:

- Atributo `String matricula;`
- Atributo `String nome;`
- Atributo `double nota;`
- Método `void validarNota(double nota);` (lança exceções personalizadas)

Crie as seguintes exceções personalizadas para tratar erros e situações específicas:

- `NotaInvalidaException`: lançada quando uma nota fornecida é inválida (menor que 0 ou maior que 10).
- `MatriculaInvalidaException`: lançada quando uma matrícula fornecida é inválida (por exemplo, uma string vazia ou com tamanho inadequado).

Implemente o método `validarNota` na classe `Aluno` para lançar a exceção `NotaInvalidaException` quando a nota fornecida for inválida. Adicione um construtor que aceite a matrícula e o nome do aluno e valide a matrícula, lançando a exceção `MatriculaInvalidaException` quando a matrícula for inválida.

Crie uma classe chamada `CadastroAlunos` com os seguintes atributos e métodos:

- Atributo `List<Aluno> alunos;`
- Método `void adicionarAluno(Aluno aluno);`
- Método `void removerAluno(String matricula);` (lança exceção personalizada)
- Método `void listarAlunos();`
- Método `Aluno buscarAluno(String matricula);` (lança exceção personalizada)

Os métodos `removerAluno` e `buscarAluno` devem lançar a exceção `AlunoInexistenteException` quando não for possível encontrar um aluno com a matrícula fornecida. Adapte a classe `CadastroAlunos` para tratar as exceções lançadas e fornecer feedback apropriado ao usuário.

Por exemplo, você pode criar um método para adicionar alunos que solicita os dados do aluno e valida a entrada:

- Método `void adicionarAlunoInterativo();`

O método `adicionarAlunoInterativo` deve solicitar os dados do aluno, como nome, matrícula e nota. Em seguida, ele deve tentar criar um objeto `Aluno` com os dados fornecidos e adicioná-lo à lista de alunos. Se ocorrerem exceções (`MatriculaInvalidaException` ou `NotaInvalidaException`), o método deve exibir uma mensagem de erro e solicitar a correção dos dados.

Exemplo de implementação do método `adicionarAlunoInterativo`:

```

1 public void adicionarAlunoInterativo() {
2     Scanner scanner = new Scanner(System.in);
3     boolean adicionadoComSucesso = false;
4
5     while (!adicionadoComSucesso) {
6         try {
7             System.out.print("Digite o nome do aluno: ");
8             String nome = scanner.nextLine();
9
10            System.out.print("Digite a matricula do aluno: ");
11            String matricula = scanner.nextLine();
12
13            System.out.print("Digite a nota do aluno: ");
14            double nota = scanner.nextDouble();
15            scanner.nextLine(); // Consumir a quebra de linha restante
16
17            Aluno aluno = new Aluno(matricula, nome);
18            aluno.validarNota(nota);
19            aluno.setNota(nota);
20            adicionarAluno(aluno);
21            adicionadoComSucesso = true;
22            System.out.println("Aluno adicionado com sucesso!");
23        } catch (MatriculaInvalidaException | NotaInvalidaException e) {
24            System.err.println("Erro: " + e.getMessage());
25            System.out.println("Por favor, insira os dados corretos.");
26        }
27    }
28 }

```